Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 278 (2026) 663–670

Procedia Computer Science

www.elsevier.com/locate/procedia

CENTERIS - International Conference on ENTERprise Information Systems / ProjMAN - International Conference on Project MANagement / HCist - International Conference on Health and Social Care Information Systems and Technologies 2025

Multiobjective Optimization Approaches for Room Allocation in University Course Timetabling☆

Bruno Antunes-Batista[a], Soumen Atta[b,*], Vítor Basto-Fernandes[a,c,1], Iryna Yevseyeva[d], Michael Emmerich[b]

[a]*Instituto Universitário de Lisboa (ISCTE-IUL), ISTAR, 1649-026 Lisboa, Portugal*
[b]*Faculty of Information Technology, University of Jyväskylä, P.O. Box 35 (Agora), Jyväskylä, 40014, Finland*
[c]*Sorbonne Université, CNRS, LIP6, 75005 Paris, France*
[d]*School of Computer Science and Informatics, Faculty of Technology, Arts, and Culture, De Montfort University, Leicester, LE1 9BH, UK*

Abstract

This study addresses the complex and time-consuming task of room allocation in university timetabling, which, despite the availability of software tools, is still predominantly performed manually due to the inefficiency of existing solutions. The adoption of such software remains limited, primarily due to complex parameter configurations and the lack of institutional customization. This paper focuses on the room allocation subproblem of the University Course Timetabling Problem (UCTP) and defines six criteria: four objectives to minimize student relocation between buildings, mismatches between requested and assigned room types, waste of room capacity and room changes between consecutive classes, and two hard constraints to ensure that room capacity requirements are respected and to prevent scheduling overlaps. Customized variants of NSGA-II, NSGA-III, and MOEA/D, tailored to the room allocation context, were developed using the jMetal framework with integrated local search. Experiments were conducted using real-world data from a university over a full academic semester, involving approximately 10,000 students, more than 26,000 scheduled class sessions, and over 120 classrooms distributed across multiple buildings with over 20 distinct room types. The customized multiobjective evolutionary algorithms were compared against an integer linear programming (ILP) formulation modeled in Pyomo and solved using Gurobi.

© 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the CENTERIS/ProjMAN/HCist conferences

Keywords: University Timetabling; Room Allocation; Metaheuristics; Multiobjective Evolutionary Algorithms; Local Search; Integer Linear Programming

☆ This work is based on the Master's thesis of Bruno Antunes-Batista, submitted at Instituto Universitário de Lisboa (ISCTE-IUL), Portugal.

* Corresponding author: Soumen Atta (soumen.s.atta@jyu.fi)
E-mail addresses: bruno_antunes@iscte-iul.pt (Bruno Antunes-Batista)., soumen.s.atta@jyu.fi (Soumen Atta)., vitor.basto.fernandes@iscte-iul.pt (Vítor Basto-Fernandes)., iryna@dmu.ac.uk (Iryna Yevseyeva)., michael.t.m.emmerich@jyu.fi (Michael Emmerich).

[1] This research was partially supported by the ISTAR projects UIDB/04466/2023 and UIDP/04466/2023.

1877-0509 © 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the CENTERIS/ProjMAN/HCist conferences

1. Introduction

Efficient academic timetable management plays a critical role in the operational success of higher education institutions. The University Course Timetabling Problem (UCTP) is recognized as a complex combinatorial optimization problem of NP-hard nature [1, 4]. The scale of the problem, combined with the limitations of existing software solutions in terms of scalability, customization, and performance, highlights the need for a more robust and institution-specific approach.

Classical methods based on Integer Linear Programming (ILP) and mixed-integer linear programming (MILP) offer precise formulations but suffer from computational intractability for large-scale instances. In contrast, population-based metaheuristics such as Genetic Algorithm (GA) and Ant Colony Optimization (ACO) have demonstrated superior performance on large and complex timetabling problems [4, 13]. Recent literature, including works by Chen et al. [4] and Bashab et al. [2], emphasizes the effectiveness of hybrid approaches that combine global metaheuristic search with local intensification strategies. These methods, particularly evolutionary algorithms [14] like NSGA-II [7], NSGA-III [6, 12], and MOEA/D [17], have shown promising results in handling multiple conflicting criteria simultaneously.

Motivated by these findings, this work proposes and compares two approaches to solving room allocation in UCTP: an ILP-based model, which aims to find optimal solutions under strict constraints, and a multiobjective evolutionary algorithm framework, implemented in *jMetal* [8, 9], which incorporates local search and constructive heuristics for initial solution generation to improve scalability and solution quality. The goal is to assess the effectiveness of both methods in meeting institutional requirements and addressing the realistic scale of the timetabling problem in a university.

This study is motivated by the need to address the UCTP through a comprehensive analysis of existing solution methods and their applicability to real-world scenarios. To guide the investigation, several research questions were formulated: (i) What are the most widely used approaches to solving the UCTP? (ii) What are their main advantages, disadvantages, and limitations? (iii) What innovations and opportunities for improvement have been proposed in the literature? (iv) What recent trends have emerged in the UCTP research? and (v) How do metaheuristic algorithms compare to exact methods such as ILP? These questions serve as a foundation for understanding the current state-of-the-art and identifying gaps that can be addressed through novel algorithmic strategies.

To answer these questions, the objectives of this work include identifying and analyzing the most widely used techniques for solving UCTP and benchmarking selected algorithms using real-world data from a university. Particular emphasis is placed on comparing the performance of multiobjective evolutionary algorithms, such as NSGA-II, NSGA-III, and MOEA/D, with ILP-based approaches, with the goal of evaluating their effectiveness in terms of scalability, solution quality, and adaptability to institutional constraints.

This paper makes two contributions: (i) *Scale-up:* we study a large-scale real-world room allocation in UCTP at a university for a semester, with more than 10,000 students, more than 26,000 classes and over 120 rooms in more than 20 types, highlighting scalability and practicality beyond previously published studies (e.g., Torres-Ovalle et al. [16]); (ii) *Multiobjective decision support:* we define six objectives to solve them using NSGA-II, NSGA-III, and MOEA/D, paired with exact solvers and employ hypervolume-driven early stopping. This allows stakeholders to evaluate different high-performing room allocations instead of a single aggregated solution.

The remainder of the paper is organized as follows. Section 2 presents the description and mathematical formulation of the problem. Section 3 outlines the proposed solution methodologies. Section 4 details the experimental setup and discusses the results and key findings. Finally, Section 5 concludes the paper and outlines directions for future research.

2. Problem Description

The problem addressed in this work focuses on the UCTP room allocation subproblem. Specifically, it assumes that the weekly timetable of classes is already defined and aims to assign classrooms to scheduled classes in a way that minimizes the violation of four soft constraints while ensuring the satisfaction of two hard constraints. This variant of the UCTP is practically relevant and computationally challenging, as it deals with optimizing room assignments within fixed temporal slots.

Each room is characterized by a set of descriptive attributes, including

- **Name:** A unique room identifier;
- **Capacity:** The total number of available seats;
- **Features:** A set of characteristics such as regular classroom, computer lab, and others;
- **Building:** The building in which the room is located.

Formally, let $\mathcal{R} = \{r_1, r_2, \ldots, r_{|\mathcal{R}|}\}$ denote the set of rooms, where each room $r_j \in \mathcal{R}$ is defined by the attributes listed above. Each scheduled class is also described by the following attributes:

- **Course:** The academic program(s) to which the class belongs;
- **Shift:** The specific shift (e.g., morning, afternoon, evening) of the class;
- **Class Name:** A unique class identifier;
- **Enrolled Students:** The number of students registered for the class;
- **Day and Time:** The scheduled day, start time, and end time of the class;
- **Requested Room Features:** The required room features for the class.

Let $\mathcal{C} = \{c_1, c_2, \ldots, c_{|\mathcal{C}|}\}$ represent the set of scheduled classes, where each class $c_i \in \mathcal{C}$ must be assigned to a suitable room $r_j \in \mathcal{R}$. The goal is to perform this assignment while respecting all hard constraints and minimizing the penalties associated with soft constraint violations.

To formally represent the room allocation optimization problem, we define a set of penalized objective functions corresponding to both soft and hard constraints. The soft constraints capture desirable properties of a good allocation and are modeled as objectives (f_1 to f_4) to be minimized. Although hard constraints must be strictly satisfied in feasible solutions, they are also expressed as penalized objective functions (f_5 and f_6) to enable uniform treatment within a multiobjective optimization framework. Next, we present the formulation of each objective function.

Student relocation between buildings. The function f_1 penalizes the allocation of consecutive classes, within the same shift, course unit, and day, to rooms located in different buildings, and its formulation is given in Eq. (1).

$$\min f_1 = \sum_{\substack{i<j \\ c_i, c_j \in \mathcal{C} \\ \texttt{shift}(c_i)=\texttt{shift}(c_j) \\ \texttt{unit}(c_i)=\texttt{unit}(c_j) \\ \texttt{day}(c_i)=\texttt{day}(c_j)}} \delta\Big(\texttt{building}(r_i) \neq \texttt{building}(r_j)\Big) \tag{1}$$

where $\delta(\cdot)$ is an indicator function that returns 1 if the condition is true, and 0 otherwise; r_i and r_j are the rooms assigned to classes c_i and c_j, respectively.

Incompatibility between requested and assigned room features. The function f_2 penalizes assignments where the functional requirements of the class (e.g., lab type) are not met by the assigned room, and it is given in Eq. (2).

$$\min f_2 = \sum_{i \in \mathcal{C}} \texttt{incompatibility}(c_i, r_i) \tag{2}$$

where $\texttt{incompatibility}(c_i, r_i)$ returns the number of requested features by class c_i that are not available in the assigned room r_i.

Waste of room capacity. The function f_3 penalizes excessive over-provisioning of classroom space, and its formulation is given in Eq. (3).

$$\min f_3 = \sum_{i \in C} \delta\left(\frac{\texttt{capacity}(r_i) - \texttt{enrolled}(c_i)}{\texttt{capacity}(r_i)} \times 100 > \theta\right) \tag{3}$$

where $\texttt{capacity}(r_i)$ is the seating capacity of room r_i; $\texttt{enrolled}(c_i)$ is the number of students enrolled in class c_i; θ is the acceptable upper limit on capacity waste (e.g., $\theta = 40\%$); $\delta(\cdot)$ returns 1 if the capacity waste exceeds θ, and 0 otherwise.

Room change for consecutive classes. This function penalizes room changes between consecutive classes of the same course unit and shift, and its formulation is given in Eq. (4).

$$\min f_4 = \sum_{\substack{i<j \\ c_i, c_j \in C \\ \texttt{unit}(c_i)=\texttt{unit}(c_j) \\ \texttt{shift}(c_i)=\texttt{shift}(c_j)}} \delta\left(\texttt{room}(c_i) \neq \texttt{room}(c_j)\right) \tag{4}$$

where $\texttt{room}(c_i)$ is the room assigned to class c_i; $\texttt{unit}(c_i)$ and $\texttt{shift}(c_i)$ are the course unit and shift of the class, respectively; $\delta(\cdot)$ returns 1 if the rooms assigned to the consecutive classes differ, and 0 otherwise.

Insufficient room capacity. The function f_5 penalizes violations where a class is assigned to a room with insufficient capacity, and it is given in Eq. (5).

$$\min f_5 = \sum_{i \in C} \delta\left(\texttt{capacity}(r_i) < \texttt{enrolled}(c_i)\right) \tag{5}$$

where $\texttt{capacity}(r_i)$ is the seating capacity of the room r_i assigned to class c_i; $\texttt{enrolled}(c_i)$ is the number of students enrolled in class c_i; $\delta(\cdot)$ is an indicator function that returns 1 if the assigned room cannot accommodate all students, and 0 otherwise.

Overlapping classes in the same room. This function, f_6, penalizes scheduling conflicts where two classes are assigned to the same room at overlapping times on the same day, and it is given in Eq. (6).

$$\min f_6 = \sum_{\substack{i<j \\ c_i, c_j \in C \\ \texttt{day}(c_i)=\texttt{day}(c_j) \\ \texttt{room}(c_i)=\texttt{room}(c_j)}} \texttt{overlap}(c_i, c_j) \tag{6}$$

where $\texttt{room}(c_i)$ is the room assigned to class c_i; $\texttt{day}(c_i)$ is the scheduled day of class c_i; $\texttt{overlap}(c_i, c_j)$ is a function that returns 1 if classes c_i and c_j have overlapping time slots, and 0 otherwise.

3. Proposed Solution Methodology

The proposed methodology for solving the room allocation of the UCTP is based on insights drawn from the literature and findings of the systematic literature review. The approach combines multiobjective optimization, hy-

brid heuristics, and exact methods. Specifically, three evolutionary multiobjective algorithms, NSGA-II [7], NSGA-III [6, 12], and MOEA/D [17], are applied due to their effectiveness in handling problems with multiple conflicting criteria. To further enhance solution quality, hybrid versions of these algorithms are developed by integrating local search techniques that intensify the search around promising individual solutions. Additionally, Integer Linear Programming (ILP) [15] is used, serving as a benchmark for evaluating the quality of metaheuristic solutions. Finally, the evolutionary process is initialized using both randomly generated solutions and heuristically constructed initial populations, with and without local refinement, to assess the impact of initial solution quality on the overall optimization performance. We aim to evaluate the impact of heuristic solution construction and local search intensification on the performance of evolutionary algorithms, and to demonstrate the robustness of hybrid methods in effectively solving complex and large-scale scheduling problems, such as the real instance obtained from a university.

The implementation of the proposed solution methodologies involved the use of NSGA-II [7], NSGA-III [6, 12], and MOEA/D [17] algorithms integrated into the jMetal [8, 9] framework, along with an ILP approach formulated using the Pyomo [3, 11] modeling language and solved using the Gurobi [10] solver. Gurobi [10], available for free for academic use, provided the required functionality. Customized variants of the evolutionary algorithms were developed with integrated local search procedures to enhance solution quality. The jMetal framework was employed to manage problem definitions, solution representations, genetic operators, and experimental evaluations. The implementation was carried out in Java and Python, reflecting the tool chain: Java for jMetal and Python for Pyomo and Gurobi solver integration.

The local search procedure was integrated into the evolutionary multiobjective algorithms using the `BasicLocalSearch` class from the jMetal [8, 9] framework. This was configured with an Integer Polynomial Mutation operator, a standard dominance comparator (`DefaultDominanceComparator` [8, 9]), and a reference to the problem model. The mutation operator was parameterized with a mutation probability of $\frac{1}{n}$, where n denotes the number of genes in a chromosome, and a distribution index of 20. In each iteration, the local search generated a single neighbor solution. If the new solution dominated the current one according to the dominance comparator, it was accepted as the replacement. This mechanism enhanced the evolutionary process by introducing targeted improvements in solution quality.

To enhance the quality of initial solutions and provide effective starting points for the evolutionary process, a constructive heuristic was implemented. This method assigns each class to a room by prioritizing the satisfaction of hard constraints. For each class $c_i \in \mathcal{C}$, the heuristic iteratively selects the first room $r_j \in \mathcal{R}$ satisfying the following conditions: $\texttt{capacity}(r_j) \geq \texttt{students}(c_i) \quad \texttt{and} \quad \neg\texttt{overlap}(c_i, r_j, D_i)$, where $\texttt{capacity}(r_j)$ is the capacity of room r_j, $\texttt{students}(c_i)$ is the number of enrolled students in class c_i, D_i is the day on which the class occurs, and $\texttt{overlap}(c_i, r_j, D_i)$ verifies whether another class already occupies room r_j at the same time on day D_i. If no feasible room is found, a room is assigned randomly as $r_j \leftarrow \texttt{rand}(\mathcal{R})$.

Instead of relying on a fixed number of iterations as the stopping criterion, the proposed method incorporates a dynamic mechanism based on the hypervolume [18] quality indicator. At each generation, the hypervolume of the current Pareto front is computed and compared against the best value observed so far. If there is no significant improvement over a predefined number of consecutive generations, the algorithm terminates early, thereby avoiding unnecessary computations. Initially, the hypervolume of the first generation is recorded as the best value. During each subsequent generation, the hypervolume of the current front is recalculated and compared with the stored best value. If the improvement exceeds a small threshold value, set to 10^{-6}, the best value is updated, and the counter for stagnation is reset. Otherwise, the counter is incremented. The algorithm halts when either the maximum number of generations is reached or the number of consecutive generations without improvement reaches a predefined limit of 100.

An ILP model is also developed in this work. The ILP model assigns classes to rooms while enforcing two hard constraints: (i) no class is assigned to a room with insufficient capacity, and (ii) overlapping classes from being scheduled in the same room at the same time is prevented. To guarantee feasibility in cases where no room satisfies the requirements, a heavily penalized dummy room is introduced. For example, a course has 59 students enrolled and requires a computer lab, but all available computer labs have a maximum capacity of only 50 seats. Without the dummy room, such cases would make the model infeasible. Moreover, the ILP model treats the two hard constraints (room capacity and room overlap) as strongly penalized objective functions (see Eq. (5) and Eq. (6)), allowing the model to generate feasible solutions even when physical limitations exist. Then, the model incorporates a weighted-sum objective function that minimizes penalties (f_1 to f_4) associated with soft constraints, and penalties (f_5 and f_6)

associated with the hard constraints, along with the dummy room usage. Equal weights are assigned to objectives f_1 through f_4, whereas high penalty weights are applied to objectives f_5, f_6, and the use of the dummy room. The penalty weights used in our experiments are fixed but can be adjusted to align with the preferences of the decision maker. The model is implemented in Python using the Pyomo [3, 11] modeling language and solved with Gurobi [10] solver.

4. Experimental Results and Discussions

The experiments were conducted on a machine equipped with an AMD Ryzen 7 7840HS processor featuring 8 physical cores and 16 logical threads, and 40 GB of RAM, running Windows 11 (64-bit). The development environment included IntelliJ IDEA for Java and Visual Studio Code for Python. The evolutionary algorithms were implemented using the jMetal 6.1 framework [8, 9], while the ILP model was developed in Python 3.12 using the Pyomo [3, 11] modeling language and solved with the Gurobi [10] optimizer.

Experimental settings. The experimental study was conducted using two distinct approaches to solve the room and timetable allocation problem: evolutionary multiobjective optimization and integer linear programming. The evolutionary approach employed three algorithms, NSGA-II [7], NSGA-III [6, 12], and MOEA/D [17], across all configurations. Four experimental setups were tested to evaluate the impact of hybridization through local search and the use of a constructive initial population. In Experiment 1, both local search and a random initial population were used. Experiment 2 excluded both local search and a constructive initial population. Experiment 3 combined both local search and a constructive initial population. Lastly, Experiment 4 used a constructive initial population without local search. Each experiment was executed with 25 independent runs to ensure robustness and comparability of the results.

Test instances. To validate the effectiveness of the proposed algorithms, a real-world problem instance was used. The real-world data provided by a university represents a full academic semester. This instance includes approximately 10,000 students, over 26,000 scheduled classes, and more than 120 rooms of over 20 distinct types, such as auditoriums, laboratories, and standard classrooms. It captures the complexity and institutional constraints of the actual room allocation problem and serves as a realistic benchmark for evaluating the scalability and robustness of the proposed methods.

Parameters of evolutionary algorithms. The configuration parameters used in the evolutionary algorithms are summarized as follows. All three algorithms, NSGA-II, NSGA-III, and MOEA/D, were executed with a population size of 200. NSGA-II and MOEA/D used a stopping criterion based on a maximum of 80,000 evaluations, while NSGA-III was limited to 400 generations. For mutation, all algorithms applied polynomial mutation with a distribution index of 20.0 and a mutation probability of one divided by the number of genes in the individual. NSGA-II and NSGA-III used the Integer Polynomial Mutation operator, whereas MOEA/D used the standard Polynomial Mutation operator. The crossover operator was Integer Simulated Binary Crossover (SBX) for NSGA-II [7] and NSGA-III, and Differential Evolution Crossover for MOEA/D [17], with corresponding parameters: a crossover probability of 0.9 and distribution index of 18.0 for the SBX [5], and Crossover Rate (CR) equal to 0.5 and Scaling Factor (F) equal to 1.0 for differential evolution. Selection in NSGA-II and NSGA-III was based on the Ranking and Crowding Distance Comparator. NSGA-III employed four reference point divisions, whereas MOEA/D used Chebychev decomposition.

Performance analysis of evolutionary algorithms. Table 1 presents the mean and standard deviation of the normalized hypervolume (NHV) [18] and generalized spread (GSPREAD) [7] for MOEA/D, NSGA-II, and NSGA-III across the four experimental configurations.

Overall, MOEA/D consistently achieved the best performance in terms of solution quality and stability across all experiments, with NHV values close to 1.000 and low standard deviations, indicating highly converged and well-distributed Pareto fronts. Its GSPREAD remained stable with slight variation, showing robustness to changes in local search and initial population strategies. We believe it achieves higher hypervolume because its decomposition uses dedicated weight vectors to cover extreme Pareto front points, and its Differential Evolution crossover creates larger moves than SBX with polynomial mutation.

NSGA-II showed significant sensitivity to the experimental configurations. Its performance improved steadily from Experiment 1 (NHV = 0.550) to Experiment 4 (NHV = 0.946), with the highest NHV observed when no hybridization

or heuristic initialization was used. The lowest GSPREAD (0.404) was obtained in Experiments 1 and 2, indicating compact and well-distributed fronts, while performance slightly declined in Experiment 3 with local search and constructive population. We believe the crowding distance assignment struggles with a higher number of objectives.

NSGA-III exhibited unstable behavior with relatively low NHV values across all experiments. Its best NHV (0.675) was recorded in Experiment 4, though with a high standard deviation, suggesting inconsistency across runs. GSPREAD values were generally moderate, with the highest value in Experiment 1 (0.636), implying better diversity when local search was applied without heuristic initialization.

In terms of runtime, Experiment 1, which used local search without a constructive population, had the longest execution time at 16 hours and 40 minutes. Experiment 3, which combined both strategies, took 15 hours and 48 minutes to complete. Experiments 2 and 4 were comparatively faster, requiring 14 hours and 10 minutes and 13 hours and 30 minutes, respectively. These results indicate that although hybridization can improve performance for some algorithms, its effectiveness and computational cost vary notably between algorithmic configurations.

Table 1. Mean and standard deviation of NHV and GSPREAD for each algorithm across the four experiments

Algorithm	**Experiment-1**		**Experiment-2**		**Experiment-3**		**Experiment-4**	
	NHV	**GSPREAD**	**NHV**	**GSPREAD**	**NHV**	**GSPREAD**	**NHV**	**GSPREAD**
MOEA/D	**0.998 ± 0.005**	**1.002 ± 0.067**	**0.999 ± 0.001**	**0.911 ± 0.127**	**0.998 ± 0.001**	**1.031 ± 0.105**	**1.000 ± 0.000**	**1.026 ± 0.098**
NSGA-II	0.550 ± 0.128	0.405 ± 0.030	0.798 ± 0.046	0.404 ± 0.026	0.800 ± 0.040	0.469 ± 0.033	0.946 ± 0.040	0.447 ± 0.032
NSGA-III	0.591 ± 0.090	0.636 ± 0.073	0.376 ± 0.107	0.371 ± 0.023	0.418 ± 0.137	0.417 ± 0.024	0.675 ± 0.219	0.455 ± 0.036

Figure 1 presents parallel coordinates plots comparing the results of four experiments. Across all experiments, MOEA/D consistently exhibits high variability, particularly in hard constraints (corresponding to the functions f_5 and f_6), while NSGA-II and NSGA-III show more stable and cohesive behavior. In Experiment 1, MOEA/D achieves extreme solutions but with global instability; NSGA-II and NSGA-III demonstrate consistency, and the ILP shows excellent constraint satisfaction but lacks solution diversity. In Experiment 2, the introduction of a heuristic-based initial population improves feasibility for all algorithms, with MOEA/D maintaining its high-performing extremes despite instability. Experiment 3 confirms previous patterns and shows limited gains from local search, as the initial population already yields high-quality solutions. Finally, Experiment 4 demonstrates MOEA/D's strong exploratory ability even without intensification, while the lack of a good initial population reduces solution diversity. Across all cases, ILP solutions remain stable and competitive on hard constraints but are limited to a single non-diverse solution.

5. Conclusion and Future Work

This study addressed the room allocation subproblem in university course timetabling, focusing on overcoming the limitations of manual scheduling and the constrained adaptability of existing software. A set of four objectives and two hard constraints was formulated to reflect practical institutional requirements. To solve this multiobjective combinatorial problem, customized NSGA-II, NSGA-III, and MOEA/D were developed using the jMetal framework with integrated local search, specifically adapted to the room allocation context. These were compared against an integer linear programming (ILP) model formulated in Pyomo and solved with Gurobi.

Experimental results on real-world data from a full academic semester of a university demonstrated that ILP consistently yielded high-quality feasible solutions, particularly for hard constraints, while the evolutionary algorithms effectively explored diverse trade-offs among conflicting objectives. MOEA/D showed notably strong convergence and spread, especially when enhanced with heuristic initialization and local search.

In the future, we plan to incorporate real-world constraints, like lecturer availability and room adjacency. Additionally, combining ILP and metaheuristics via hybrid or decomposition frameworks could enhance scalability and solution quality for large-scale timetabling problems. Furthermore, future work will have to prioritize constructing a broader set of benchmark data for large-scale problem solving using a posteriori multiobjective optimization to assess method reliability across various problem instances.

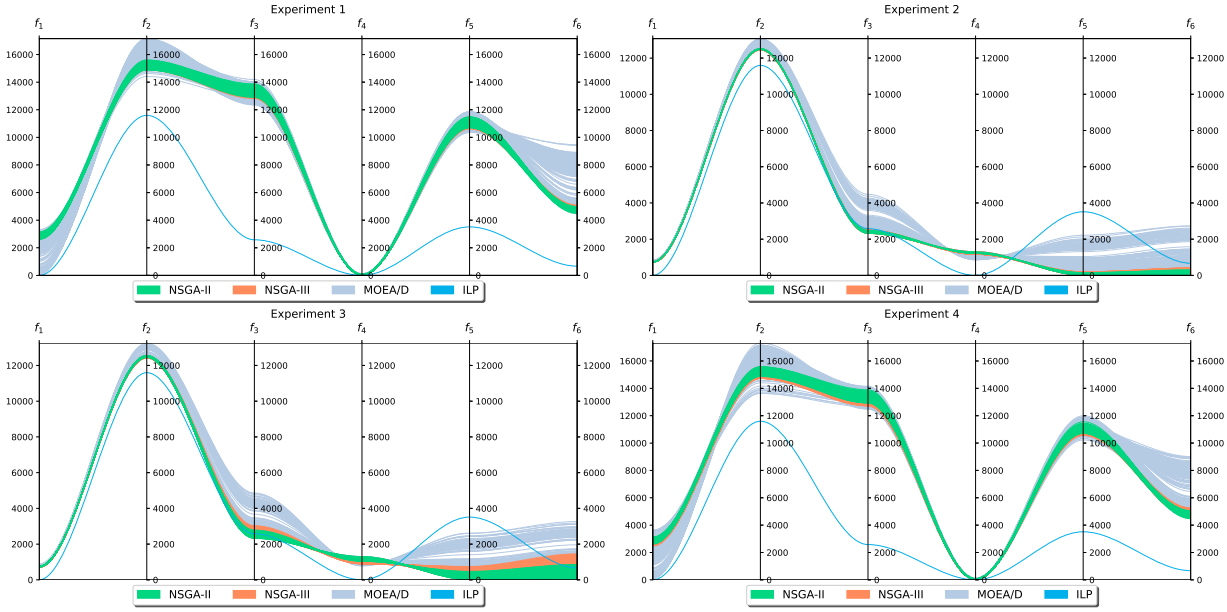

Fig. 1. Parallel coordinates plots comparing the four experimental configurations

References

[1] Babaei, H., Karimpour, J., Hadidi, A., 2015. A survey of approaches for university course timetabling problem. Computers & Industrial Engineering 86, 43–59.

[2] Bashab, A., Ibrahim, A.O., Tarigo Hashem, I.A., Aggarwal, K., Mukhlif, F., Ghaleb, F.A., Abdelmaboud, A., 2023. Optimization techniques in university timetabling problem: Constraints, methodologies, benchmarks, and open issues. Computers, Materials & Continua 74, 6461–6484.

[3] Bynum, M.L., Hackebeil, G.A., Hart, W.E., Laird, C.D., Nicholson, B.L., Siirola, J.D., Watson, J.P., Woodruff, D.L., 2021. Pyomo–optimization modeling in python. volume 67. Third ed., Springer Science & Business Media.

[4] Chen, M.C., Goh, S.L., Sabar, N.R., Kendall, G., et al., 2021. A survey of university course timetabling problem: perspectives, trends and opportunities. IEEE Access 9, 106515–106529.

[5] Deb, K., Agrawal, R.B., 1995. Simulated binary crossover for continuous search space. Complex Systems 9, 115–148.

[6] Deb, K., Jain, H., 2013. An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I: Solving Problems With Box Constraints. IEEE Transactions on Evolutionary Computation 18, 577–601.

[7] Deb, K., Pratap, A., Agarwal, S., Meyarivan, T., 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation 6, 182–197.

[8] Durillo, J.J., Nebro, A.J., 2011. jMetal: A Java framework for multi-objective optimization. Advances in engineering software 42, 760–771.

[9] Durillo, J.J., Nebro, A.J., Alba, E., 2010. The jmetal framework for multi-objective optimization: Design and architecture, in: IEEE Congress on Evolutionary Computation, IEEE. pp. 1–8.

[10] Gurobi Optimization, LLC, 2024. Gurobi Optimizer Reference Manual. URL: https://www.gurobi.com.

[11] Hart, W.E., Watson, J.P., Woodruff, D.L., 2011. Pyomo: modeling and solving mathematical programs in python. Mathematical Programming Computation 3, 219–260.

[12] Jain, H., Deb, K., 2013. An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point Based Nondominated Sorting Approach, Part II: Handling Constraints and Extending to an Adaptive Approach. IEEE Transactions on Evolutionary Computation 18, 602–622.

[13] Pandey, J., Sharma, A., 2016. Survey on university timetabling problem, in: 2016 3rd International conference on computing for sustainable global development (INDIACom), IEEE. pp. 160–164.

[14] Pétrowski, A., Ben-Hamida, S., 2017. Evolutionary algorithms. volume 9. John Wiley & Sons.

[15] Schrijver, A., 1998. Theory of linear and integer programming. John Wiley & Sons.

[16] Torres-Ovalle, C., Montoya-Torres, J.R., Quintero-Araújo, C.L., Sarmiento-Lepesqueur, A., Castilla-Luna, M., 2014. University course scheduling and classroom assignment. Ingeniería y Universidad 18, 59–75.

[17] Zhang, Q., Li, H., 2007. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. IEEE Transactions on Evolutionary Computation 11, 712–731.

[18] Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C.M., Da Fonseca, V.G., 2003. Performance assessment of multiobjective optimizers: An analysis and review. IEEE Transactions on Evolutionary Computation 7, 117–132.